

Guia Prático de Linguagem SQL

Transição do MySQL para o PostgreSQL (DDL & DML)

1 Alterando a Estrutura da Tabela (DDL – ALTER TABLE e DROP TABLE)

No PostgreSQL, a manipulação de estruturas de tabelas possui sintaxe padrão ANSI SQL estrita. Algumas funcionalidades específicas do MySQL (como AFTER, FIRST e MODIFY COLUMN) funcionam de maneira diferente ou exigem abordagens distintas no Postgres.

1.1 Adicionar uma nova coluna

A sintaxe básica para adicionar colunas no PostgreSQL é bastante direta e segue o padrão SQL ANSI:

```
alter table pessoas
add column profissao varchar(10);
```

1.2 Remover uma coluna

Para apagar uma coluna existente:

```
alter table pessoas
drop column profissao;
```

1.3 Posicionamento de colunas (AFTER e FIRST)

⚠ Atenção Pedagógica (Diferença para o MySQL):

O PostgreSQL **não suporta** as cláusulas AFTER ou FIRST. No modelo relacional formal, a ordem física das colunas em uma tabela é irrelevante.

- Se a ordem das colunas for importante para visualização em aula, recomenda-se organizar a ordem dos campos no comando SELECT ou criar uma VIEW.
- Para adicionar a coluna, basta executar o comando padrão de inserção:

```
-- Adiciona a coluna (ficará posicionada ao final da estrutura física)
alter table pessoas
add column profissao varchar(10);

alter table pessoas
add column codigo int;
```

1.4 Alterando o tipo de dados de uma coluna

No MySQL utiliza-se MODIFY COLUMN. No PostgreSQL, utiliza-se ALTER COLUMN ... TYPE:

```
alter table pessoas
alter column profissao type varchar(20);
```

💡 Dica de Aula (PostgreSQL):

Se a coluna já contiver dados que precisem de conversão explícita de tipo (por exemplo, converter `text` para `integer`), adicione a cláusula `USING`:

```
alter table pessoas alter column idade type int using (idade::integer);
```

1.5 Renomeando uma coluna

No MySQL utiliza-se `CHANGE COLUMN`. No PostgreSQL, a alteração do nome é isolada da alteração de tipo com a cláusula `RENAME COLUMN`:

```
alter table pessoas
rename column profissao to prof;
```

1.6 Renomeando uma tabela

A sintaxe é idêntica ao padrão ANSI:

```
alter table pessoas
rename to gafanhotos;
```

1.7 Criando uma nova tabela

No PostgreSQL:

1. O tipo `unsigned` não existe. Utiliza-se restrições do tipo `CHECK` se necessário garantir apenas números positivos.
2. O tipo `year` não existe. Utiliza-se `integer` ou o tipo nativo de data `date`.
3. O parâmetro `default charset` é omitido, pois o PostgreSQL define o encoding no nível do banco de dados (geralmente UTF8 por padrão).

```
create table if not exists cursos (
    nome varchar(30) not null unique,
    descricao text,
    carga int check (carga >= 0),
    totaulas int check (totaulas >= 0),
    ano int default 2016
);
```

1.8 Adicionando chave primária

Para gerar colunas autoincrementáveis ou id no PostgreSQL, o ideal é utilizar o tipo `SERIAL` (ou a sintaxe moderna `GENERATED ALWAYS AS IDENTITY`):

```
-- Adicionando a coluna idcurso
alter table cursos
add column idcurso serial;

-- Definindo como chave primária
alter table cursos
add primary key (idcurso);
```

2 Manipulando Linhas (DML – UPDATE, DELETE, TRUNCATE)

2.1 Atualizando dados (UPDATE)

A sintaxe de atualização simples é idêntica:

```
update cursos
set nome = 'HTML5',
where idcurso = 1;
```

Nota sobre Tipos de Dados:

Se a coluna `idcurso` for numérica (`int/serial`), é recomendável passar o valor numérico sem aspas (1 em vez de '1'), respeitando a tipagem forte do PostgreSQL.

Atualizando múltiplos campos simultaneamente:

```
update cursos
set nome = 'PHP', ano = 2015
where idcurso = 4;
```

```
update cursos
set nome = 'Java', carga = 40, ano = 2015
where idcurso = 5;
```

2.2 Limitação de linhas em atualizações e remoções (LIMIT)

⚠ Atenção Pedagógica (Diferença Crucial):

No MySQL, permite-se o uso de `LIMIT 1` diretamente no final de um `UPDATE` ou `DELETE`. O PostgreSQL não suporta a cláusula `LIMIT` diretamente nos comandos `UPDATE` ou `DELETE`.

Para restringir uma alteração a apenas uma linha no PostgreSQL (caso a busca retorne mais de um registro e você precise afetar apenas o primeiro), utiliza-se uma subconsulta com a chave primária (`ctid` ou `idcurso`):

```
-- Exemplo de atualização limitada a 1 registro via Subquery
update cursos
set carga = 50
where idcurso in (
    select idcurso
    from cursos
    where ano = 2015
    limit 1
);
```

2.3 Apagando linhas (DELETE)

Removendo um registro específico por ID:

```
delete from cursos
where idcurso = 8;
```

Removendo registros com base em uma condição:

```
delete from cursos
where ano = 2018;
```

Tradução do comando: Este comando apaga todas as linhas da tabela `cursos` em que o valor armazenado na coluna `ano` seja igual a 2018.

2.4 Esvaziando a tabela (TRUNCATE)

O comando TRUNCATE limpa todos os dados da tabela de forma rápida e eficiente. No PostgreSQL, possui suporte a cláusulas adicionais como RESTART IDENTITY para zerar os contadores de sequências autoincrementáveis (SERIAL):

```
-- Esvazia a tabela e reinicia os contadores de autoincremento
truncate table cursos restart identity;
```